

Natural Language Processing

Virtsudo

May 17, 2026

1 Dataset Characterization

In this section, we analyzed the label distribution across the train and test sets, examined the frequency of a single bash command `echo` across different MITRE tactics, and explored the number of bash words per session using the *Estimated Cumulative Distribution Function (ECDF)*.

Explore the labels. As shown in Fig. 1, there are seven unique MITRE tactic tags: *Defense Evasion*, *Discovery*, *Execution*, *Impact*, *Persistence*, *Not Malicious Yet*, and *Other*. The train set contains significantly more labeled words than the test set. The three dominant classes are *Discovery*, *Execution*, and *Persistence*, while the remaining classes are minorities with fewer than **1000** words each. The most frequent tag is *Discovery* (**6000** in train, **3500** in test), while *Impact* and *Other* are the least represented. Such class imbalance may cause the model to underperform on minority classes during training.

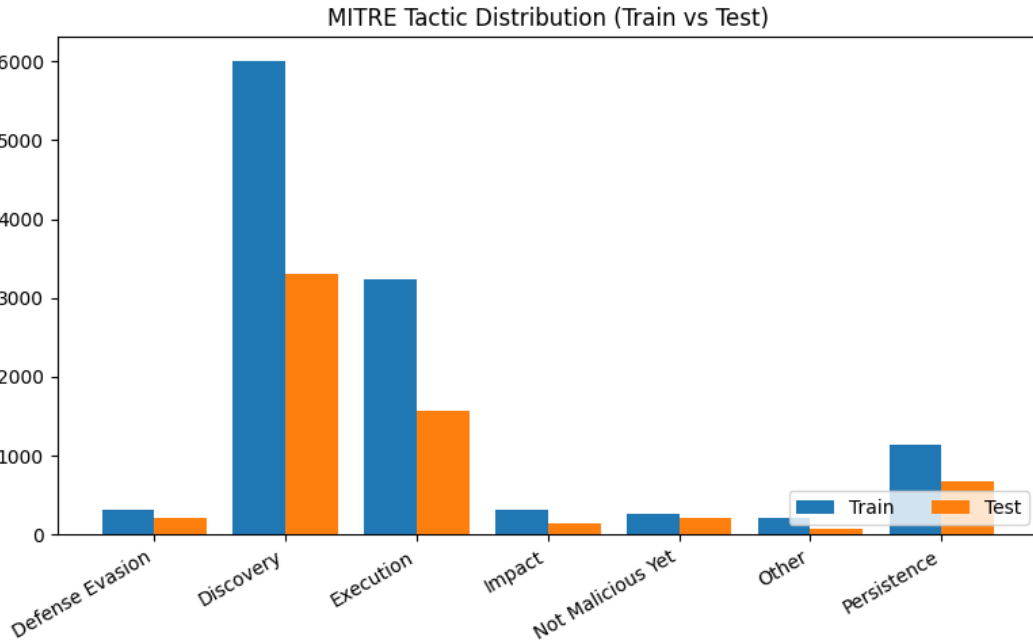


Figure 1: Train and test sets MITRE Tactics Distribution, showing class imbalance between labels

Analysis of a single bash command. To further investigate the datasets, we analyzed the `echo` command in more detail. In total, six tags were assigned to `echo` in the train set (*Persistence*, *Execution*, *Discovery*, *Not Malicious Yet*, *Impact*, *Other*), while the test set contained five (*Persistence*, *Execution*, *Discovery*, *Not Malicious Yet*, *Defense Evasion*). As shown in Fig. 2, *Persistence* was the most frequent tag, followed closely by *Discovery* and *Execution*. The remaining tags appeared fewer than **20** times each. As discovered, a single command was used for different purposes, we analyzed the two most frequent tags to explain why this occurred. The answer lies in the context of the session. As shown below, the same `echo` command receives different tags depending on its surrounding commands.

Persistence example:

```
... echo root:bFAy5nFET5hq | chpasswd | bash ...
```

Execution example:

```
... echo IyEvYmluL2Jhc2gK... | base64 --decode | bash ;
```

In the first example, `echo` was used to change the `root` password via `chpasswd`, granting the attacker persistent access even after a system reboot, which justifies the *Persistence* tag. In the second example, `echo` decoded and immediately executed a base64-encoded script via `bash`, making it a direct *Execution* action. This demonstrates that the same bash word can carry different attacker intent depending on context, which makes the NER task non-trivial and justifies the use of a context-aware model like BERT.

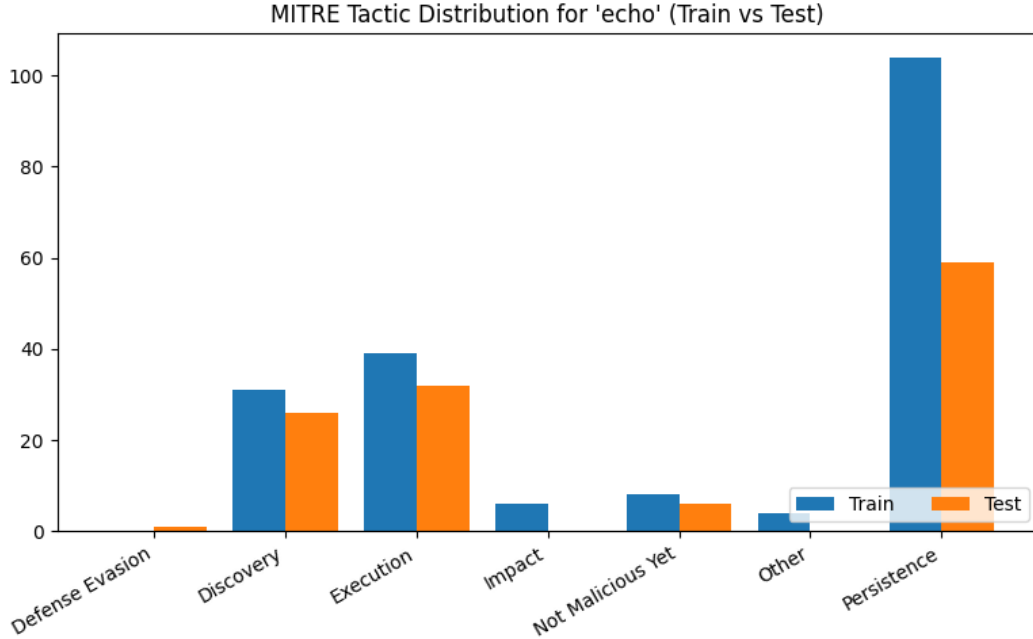


Figure 2: Train and test sets MITRE Tactics Distribution only for `echo` bash command

Explore the bash words. The last step of the analysis was the bash words distribution per session. The average number of bash words per session was **45.72** in the train set and **57.38** in the test set. As shown in Fig. 3, both sets had nearly the same distribution. The curve rose steeply until around **30** words, indicating that the majority of sessions were short, while it gradually reached 1.0 near **200** words, confirming that very long sessions were rare.

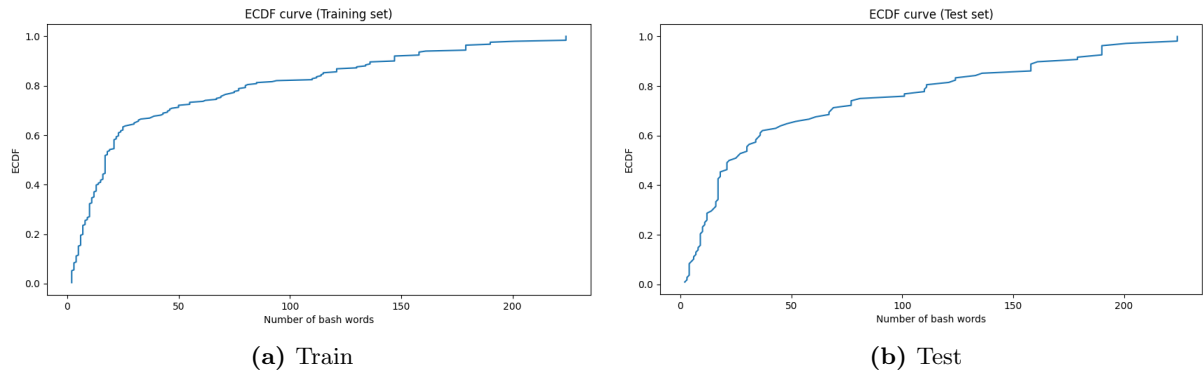


Figure 3: ECDF of bash words per session for train and test sets

2 Tokenization

In this section, we experimented with and compared two tokenizers: *BERT-base* and *UnixCoder-base*. First, we tokenized a small sample of nine bash commands, then tokenized the full training corpus, and finally applied word truncation to long tokens before re-tokenizing.

2.1 Full Corpus Tokenization

Sample tokenization.

To understand how each tokenizer processed *SSH* commands, we first tokenized a small set of nine bash commands and compared the results. As shown in Table 1, *BERT* and *UnixCoder* tokenized the same commands differently. For example, `chpasswd` was split into **3** tokens by *BERT*, while *UnixCoder* produced only **2** tokens. This difference occurs because they use different encoding strategies: *BERT* uses *WordPiece* encoding, while *UnixCoder* uses *Byte-Pair Encoding* (BPE). Overall, *BERT* produced a higher token-to-word ratio of **3.78** compared to **3.56** of *UnixCoder* on the nine sample commands. Some bash commands, such as `cat`, `top` and `grep`, were kept as a single token by both tokenizers, since they are short and common enough to exist as whole words in both vocabularies.

Command	BERT	UnixCoder
<code>cat</code>	1	1
<code>shell</code>	1	1
<code>echo</code>	1	1
<code>top</code>	1	1
<code>chpasswd</code>	3	2
<code>crontab</code>	3	3
<code>wget</code>	2	2
<code>busybox</code>	2	2
<code>grep</code>	2	1

Table 1: Token counts per command for BERT and UnixCoder tokenizers

Full corpus tokenization. Next, we tokenized the full training corpus with both tokenizers. *UnixCoder* produced a higher average number of tokens per session (**409**) compared to *BERT*, which produced only **178**. The main reason for this large gap is that *BERT* assigns unknown strings to a single [UNK] token, losing information, while *UnixCoder* splits such strings into byte pairs, preserving more useful information. The maximum number of tokens per session was **1889** for *BERT* and **28920** for *UnixCoder*, caused by very long bash words such as `base64`-encoded payloads. At a limit of 512 tokens, **24** sessions were truncated by *BERT* and **29** by *UnixCoder*. Both tokenizers identified the same session as the longest, containing **134** bash words, since it included URLs, `base64`-encoded strings and long directory paths, which split into many tokens.

2.2 After Word Truncation

Bash words longer than **30** characters were truncated, since **98%** of words are shorter than this threshold.

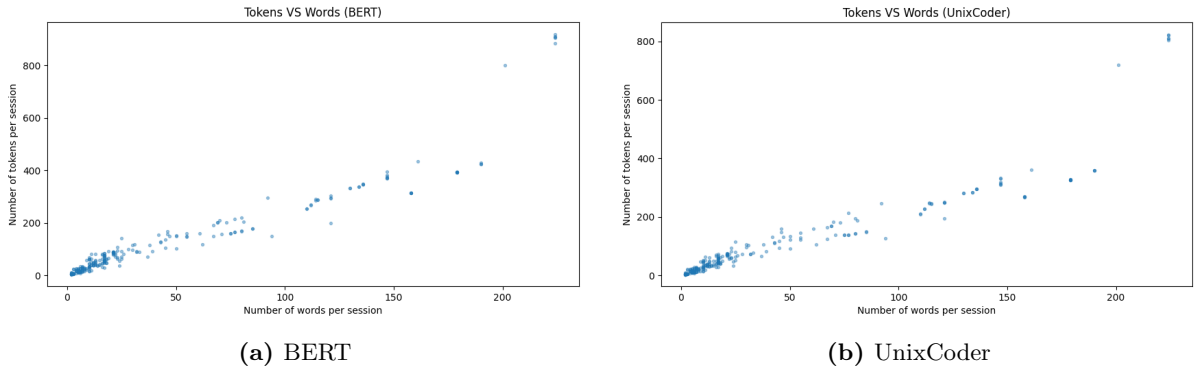


Figure 4: Tokens vs words per session scatter

After re-tokenization, the average number of tokens per session dropped significantly: *BERT* produced **128** tokens on average, while *UnixCoder* produced **110**. The maximum number of tokens per session also decreased to **920** for *BERT* and **824** for *UnixCoder*. As shown in Fig. 4, both tokenizers showed a linear relationship between the number of words and tokens per session. However, *UnixCoder* had a lower slope, indicating a better token-to-word ratio of **2.77** compared to **3.23** of *BERT*, since it no longer needed to split long unknown strings into many subwords. Finally, the number of truncated sessions at the 512 token limit dropped to only **6** for both tokenizers.

3 Model Training

In this section, we fine-tuned and compared multiple model variants. Before fine-tuning, the same word truncation preprocessing from Section 2 was applied, and the data was split into train and validation sets. We fine-tuned *BERT* in two versions: one with pre-trained weights and one with randomly initialized weights (naked). Next, we fine-tuned *UnixCoder* and compared it against the best *BERT* variant. Finally, we applied frozen fine-tuning strategies to the best model, training only the last two encoder layers with the classification head, and then only the classification head alone.

3.1 BERT pre-trained vs naked

We fine-tuned two *BERT* models with the same configuration, the only difference being the weight initialization. The first model used pre-trained *BERT*-base weights, while the second was randomly initialized (naked). As shown in Table 2, the pre-trained variant outperformed the naked one across all metrics. The naked *BERT* performed poorly, reaching a *Recall* of only **0.42** and an *F1-macro* of **0.45**, while the pre-trained model achieved **0.58** and **0.64** respectively. The average fidelity further confirmed this gap (**0.83** vs **0.71**). The poor performance of the naked model was expected, as **359** labeled samples are insufficient to learn useful representations from scratch. Pre-trained weights already encode general language knowledge, giving the model a strong starting point even with limited data.

BERT models		
Metrics	Pre-trained	Naked
Accuracy	0.85	0.73
Precision	0.82	0.59
Recall	0.58	0.42
F1-macro	0.64	0.45
Fidelity (avg.)	0.83	0.71

Table 2: *BERT* models comparison by all metrics and average fidelity

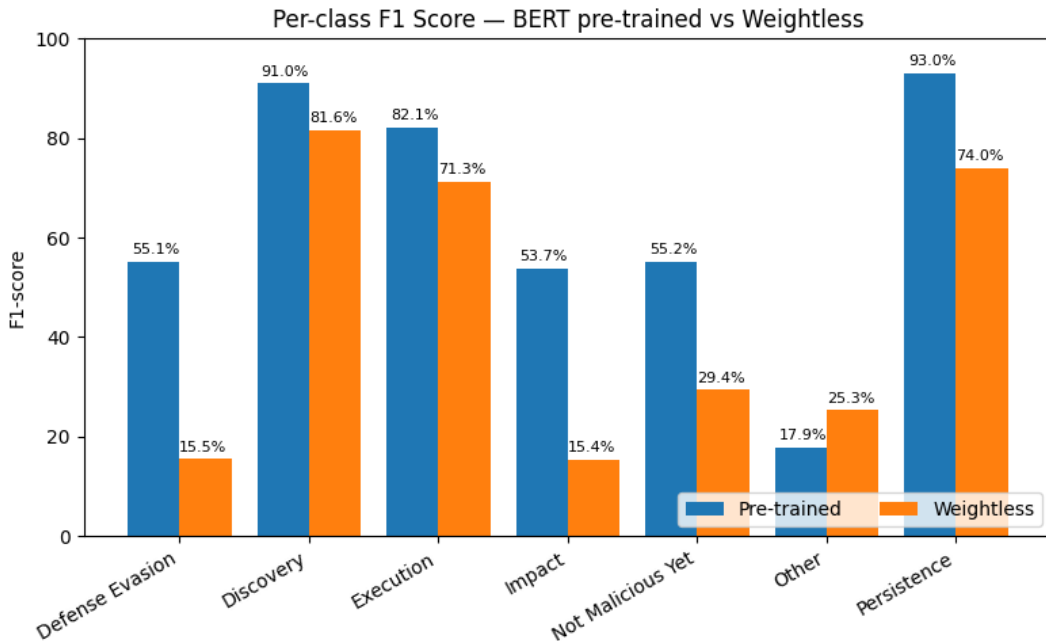


Figure 5: Pre-trained and Naked *BERT* models per-class *F1-score*, showing dominance of pre-trained model over naked

While the macro *F1-score* provided an aggregated view, the per-class *F1-score* revealed where each model struggled. As shown in Fig. 5, the pre-trained model outperformed the naked one on almost every class, except *Other*, where the naked model scored slightly higher (**25.3%** vs **17.9%**). The naked *BERT* performed reasonably only on the three majority classes, while failing on the remaining minority classes with scores below **30%**, confirming that limited training data severely impacts performance on underrepresented classes.

3.2 BERT vs UnixCoder

We fine-tuned *UnixCoder* with the same configuration as the pre-trained *BERT* model, the only differences being the pre-training corpus and the tokenizer. As shown in Table 3, *UnixCoder* outperformed *BERT* on most metrics. Although *UnixCoder* achieved a slightly lower *Precision* (**0.76** vs **0.82**), it improved significantly on *Recall* (**0.69** vs **0.58**) and *F1-macro* (**0.72** vs **0.64**), confirming the hypothesis that domain-specific pre-training is beneficial. The average fidelity also improved slightly (**0.87** vs **0.83**). This was expected, since unlike *BERT*, which was pre-trained on natural language text only, *UnixCoder* was pre-trained on code including Bash scripts, giving it stronger prior knowledge of SSH command structure and syntax.

Since the label distribution was imbalanced, we compared the per-class *F1-score* of both models, as shown in Fig. 6. *UnixCoder* dominated on almost all classes, with the only exception being *Impact*, where *BERT* scored slightly higher, though the difference was negligible. The advantage of *UnixCoder* was most visible on the *Other* class, where it achieved **51.3%** compared to only **17.9%** by *BERT*, more than double the score. This further confirms that domain-specific pre-training helps the model better handle difficult and minority classes.

Metric	BERT	UnixCoder
Accuracy	0.85	0.87
Precision	0.82	0.76
Recall	0.58	0.69
F1-macro	0.64	0.72
Fidelity (avg.)	0.83	0.87

Table 3: *BERT* and *UnixCoder* models comparison by all metrics and average fidelity

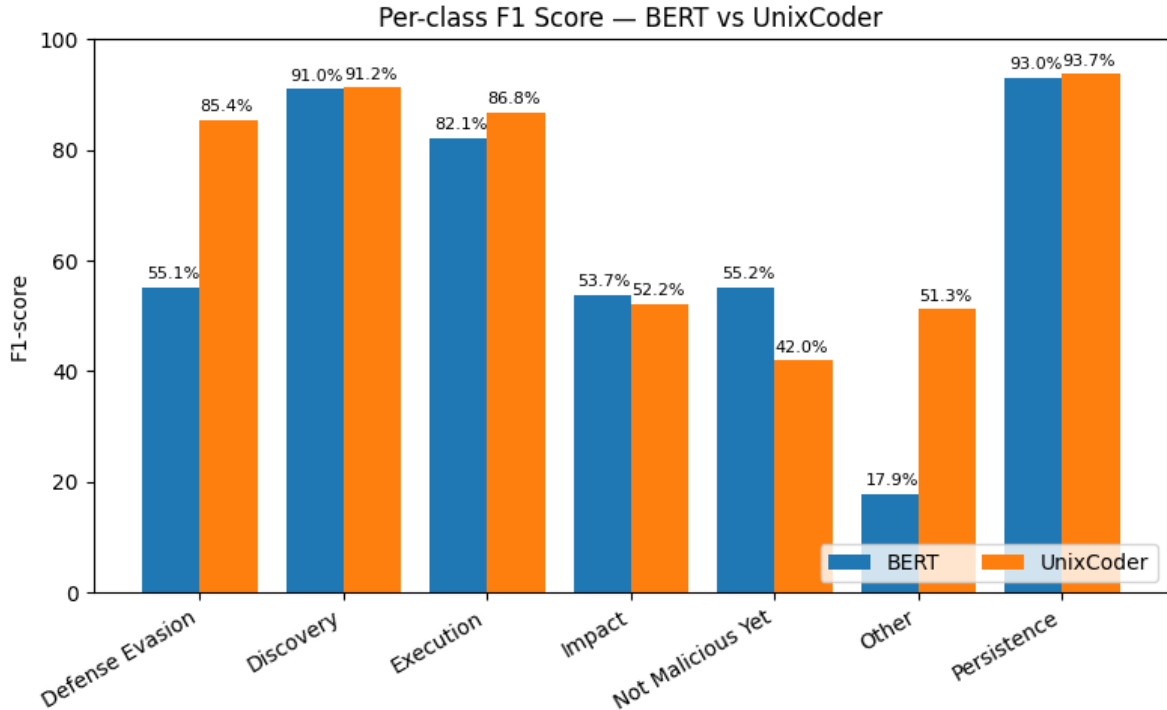


Figure 6: *BERT* and *UnixCoder* per-class *F1-score*, showing an advantage of *UnixCoder* over *BERT* almost on each class

3.3 Frozen fine-tuning

Finally, we applied frozen fine-tuning strategies to the best model (*UnixCoder*). Two variants were trained with different numbers of trainable parameters:

- the classification head and the last two encoder layers only (frozen model)
- the classification head only (class head model)

Both variants were compared against each other and against the fully fine-tuned *UnixCoder* model.

3.3.1 Last Two Layers + Classification Head

While the total number of parameters was **125.344.519**, only **14.181.127** were trainable in this variant. All metrics dropped compared to the fully fine-tuned *UnixCoder*. The macro *F1-score* fell to **0.53** (vs **0.72**), average fidelity to **0.79** (vs **0.87**), and *Recall* dropped most significantly to **0.49** (vs **0.69**). However, training time reduced from **1 minute 38 seconds** to only **45 seconds**. The performance drop occurred because freezing most encoder layers prevented the model from fully adapting its representations to the SSH domain.

3.3.2 Classification Head Only

From **125.344.519** total parameters, only **5.383** were trainable in the class head model. This variant performed worse than the frozen model across all metrics. Although *Accuracy* reached **0.59** and average fidelity **0.56**, the macro *F1-score* of only **0.22** revealed the true failure of this model. Since the dataset was imbalanced, accuracy alone is misleading, and the low macro *F1-score* confirmed that the model could not learn meaningful patterns with a completely frozen encoder.

As shown in Fig. 7, both frozen variants could only classify the majority classes, while minority classes were largely misclassified. The frozen model scored poorly on minority classes, while the class head model scored **zero** on all of them. This confirms a clear trend: the fewer trainable encoder layers, the worse the performance, since the small dataset is insufficient to compensate for frozen representations. Full encoder fine-tuning is therefore essential for domain-specific tasks like SSH NER.

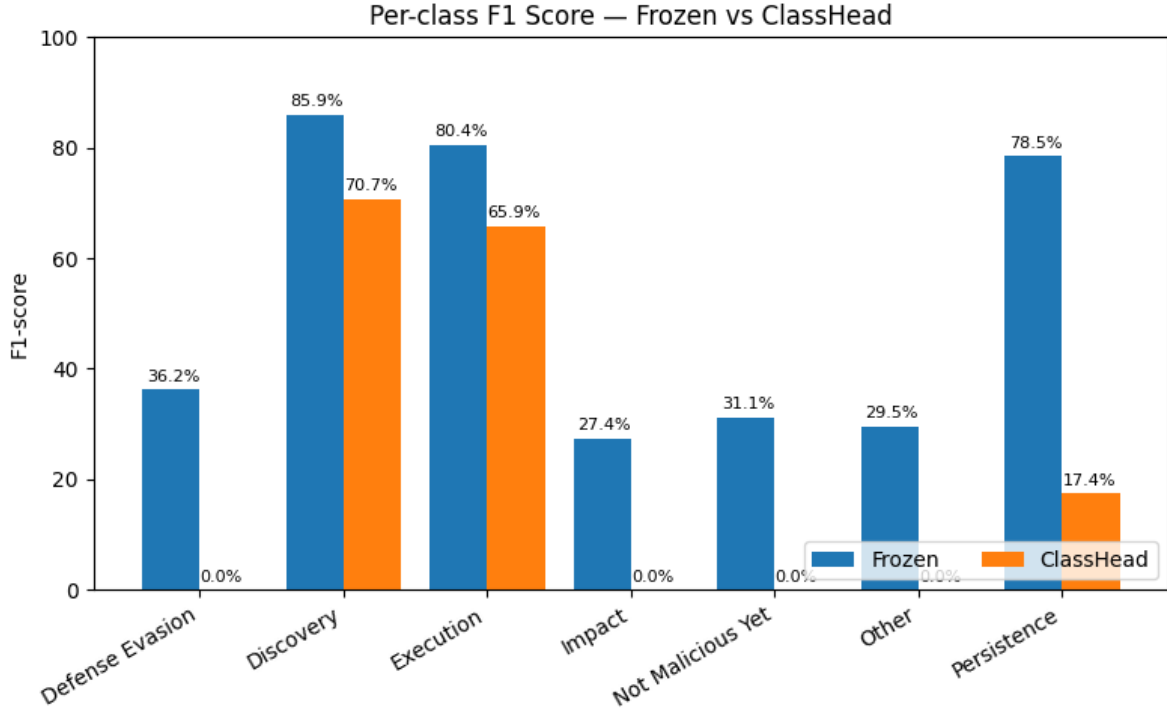


Figure 7: Frozen and Class Head models per-class *F1-score*, showing total failure of class head model

3.4 Summary

Finally, Table 4 summarizes all model variants and their metrics. The best performing model was the fully fine-tuned *UnixCoder*, thanks to its code-aware pre-training. The second best was the pre-trained *BERT*, which confirmed that pre-trained weights are essential when labeled data is scarce. For the same reason, the naked *BERT* and the class head only *UnixCoder* both failed to achieve acceptable performance. The *UnixCoder* variants also revealed a clear trend: the fewer trainable encoder layers, the worse the performance, particularly on an imbalanced dataset.

Model	Type	Accuracy	Precision	Recall	F1-macro	Fidelity (avg)
BERT	Pre-trained	0.85	0.82	0.58	0.64	0.83
	Naked	0.73	0.59	0.42	0.45	0.71
UnixCoder	Pre-trained	0.87	0.76	0.69	0.72	0.87
	Last 2 enc + head	0.79	0.72	0.49	0.53	0.79
	Head only	0.59	0.22	0.24	0.22	0.56

Table 4: All model variants comparison by metrics and average fidelity

4 Inference

In this section, we applied the best model from the previous experiments to the unlabeled inference dataset. We preprocessed the data using the same pipeline as before, then collected the model predictions for analysis. We focused on four bash commands and examined their predicted MITRE tags, followed by a fingerprint analysis to identify recurring attack patterns over time.

4.1 Commands Tag Frequency

The *UnixCoder* (pre-trained) model, selected as the best model in Section 3, was applied to the unlabeled inference dataset using the same preprocessing pipeline as before. Since no labels were available, predictions were collected for analysis rather than metric computation. Four bash commands were selected for predicted tag analysis: `cat`, `grep`, `echo`, and `rm`.

As shown in Table 5 and Table 6, some commands were dominated by a single tag, while others were distributed across multiple tags. Specifically, `cat` and `grep` were almost exclusively assigned to *Discovery* (99.92% and 99.96% respectively), which makes semantic sense since their primary purpose is reading and searching files — both reconnaissance actions. The `echo` command was distributed across three tags: *Persistence* (50.61%), *Discovery* (34.79%), and *Execution* (14.22%), consistent with the context-dependent behavior discussed in Section 1. However, `rm` was mostly assigned to *Discovery* (95.45%), while ideally it should be *Defense Evasion*, since deleting logs is a common technique to hide attacker activity. This suggests the model still struggles with context-dependent minority class predictions.

Command	Discovery	Execution	Persistence	Def. Evasion	Other	Not Mal.
<code>cat</code>	860900	664	0	0	0	1
<code>grep</code>	995988	0	386	0	0	0
<code>echo</code>	266654	108971	387857	0	2624	332
<code>rm</code>	329286	1397	5443	8846	0	0

Table 5: Predicted tag frequency per command

Command	Discovery	Execution	Persistence	Def. Evasion	Other	Not Mal.
<code>cat</code>	99.92%	0.08%	0.00%	0.00%	0.00%	0.00%
<code>grep</code>	99.96%	0.00%	0.04%	0.00%	0.00%	0.00%
<code>echo</code>	34.79%	14.22%	50.61%	0.00%	0.34%	0.04%
<code>rm</code>	95.45%	0.40%	1.58%	2.56%	0.00%	0.00%

Table 6: Predicted tag percentage per command

To further validate the predictions, one session example per unique command-tag pair was examined. The `cat` command tagged as *Discovery* appeared in sessions like "... `cat /proc/mounts ; cat /dev/shm ...`", where it was used to read system files for reconnaissance. When tagged as *Execution*, `cat` appeared alongside password-changing commands like "... `cat /proc/cpuinfo | grep name ; echo root:XP3... | chpasswd ...`", where the session context shifted toward active exploitation. The `grep` command was almost exclusively *Discovery*, appearing in sessions like "... `grep name |`

`wc -l ...`” to extract system information. The `echo` command tagged as *Persistence* appeared in sessions writing credentials to files, while *Execution* examples involved base64-decoded script execution, consistent with Section 1. Finally, `rm` tagged as *Defense Evasion* appeared in sessions like “... `rm -rf /var/tmp/.var* ...`”, removing temporary files to erase attacker traces.

4.2 Fingerprint Analysis

The fingerprint is a unique sequence of predicted MITRE tags representing the behavioral pattern of a session, while the fingerprint ID is a unique identifier used to track and differentiate these patterns over time. As shown in Fig. 8, fingerprint IDs grew continuously over the collection period, indicating that new attack patterns kept appearing. Low fingerprint IDs appeared consistently across almost all dates, suggesting recurring attack campaigns that persisted throughout the entire collection period. Fingerprints associated with large numbers of sessions were highlighted by lighter colors (green, yellow) and larger circle sizes. In early December, a sharp increase in new fingerprints was observed, accompanied by large yellow circles, indicating a suspicious surge in attack activity. This approach demonstrated that NLP-based tagging can effectively reveal attack patterns at scale across **170k** sessions, which would be practically impossible to analyze manually in a cybersecurity context.

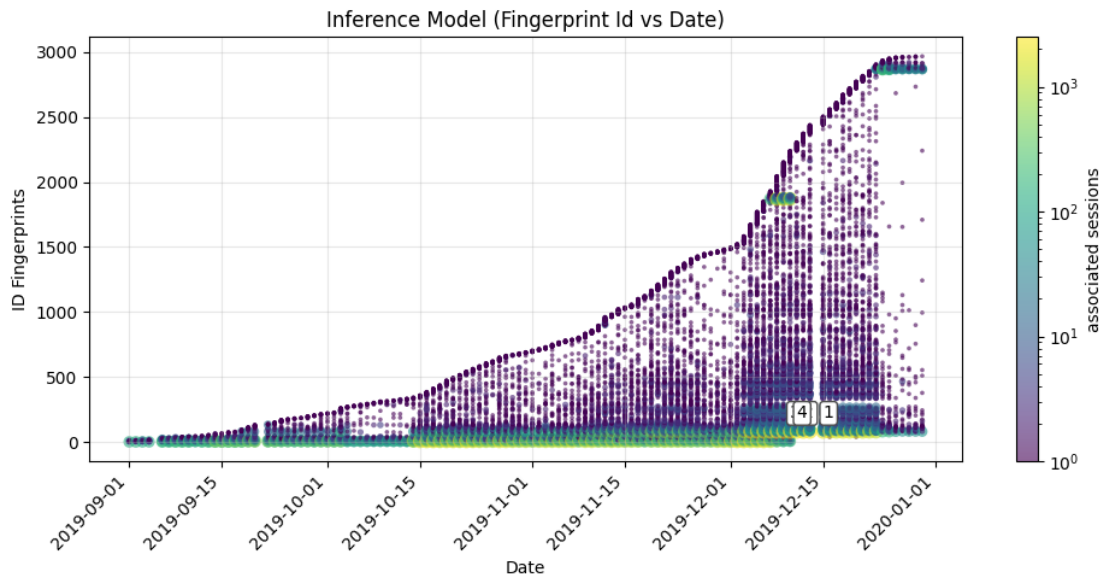


Figure 8: Fingerprint ID over time, showing recurring and emerging attack patterns